

Online Appendix: Dataset Construction Details

Peerachai Banyongrakkul, Mansooreh Zahedi, Christoph Treude, Haoyu Gao, Patanamon Thongtanunam

This appendix provides detailed descriptions of the dataset construction process described in Section IV of the main manuscript, including reuse signature extraction, query strategies, static analysis design, release-based filtering criteria, and dataset validation. It is intended to serve as a practical guideline for reproducing our dataset and to improve the transparency and reproducibility of our study.

Appendix structure. The appendix is organized as follows: Appendix A describes downstream repository identification, including reuse signature extraction and GitHub query strategies. Appendix B presents the static validation process and repository-PTM mapping, including false positive filtering. Appendix C details the final filtering on releases, including release-based filtering criteria and semantic versioning classification.

Appendix A Downstream Repository Identification

A.1 Reuse Signature Extraction

To identify PTM reuse in source code, we construct a set of *reuse signatures*, defined as code patterns that indicate PTM loading or instantiation. Following prior work [1], each signature consists of two components: (1) a library import statement (e.g., `transformers`), and (2) a model-loading function or method call (e.g., `from_pretrained()`).

Example. Listing 1 illustrates a typical reuse signature. The import statement identifies the library (`transformers`), while the call to `from_pretrained()` loads a specific PTM (`bert-base-uncased`). Together, they form a reuse signature `(transformers, from_pretrained())`.

```
from transformers import AutoModel

model = AutoModel.from_pretrained("bert-base-uncased")
```

Listing 1. Example of a reuse signature combining a library import and a model-loading call.

To ensure coverage and correctness, we systematically reviewed the official Hugging Face documentation¹ to extend the set of signatures identified by [1]. The documentation catalogs libraries that support PTM reuse (e.g., `transformers`, `diffusers`, `timm`, and `peft`). The first author and a co-author independently examined the API reference sections of each library to identify relevant classes, methods, and function calls associated with PTM loading and instantiation. For example, within the API documentation of the `transformers`

library², we identify the function call `from_pretrained()`, as illustrated in Figure 1, which is used to load a model such as `meta-llama/Llama-2-7b-hf`. Each identified pattern is recorded as a pair of the form `(library, method)`, e.g., `(transformers, from_pretrained())`.

The extracted signatures from both reviewers were consolidated and reconciled through discussion to resolve discrepancies. The final set of reuse signatures was reviewed and agreed upon by all authors. This process resulted in 355 distinct reuse patterns spanning 26 Python libraries. The complete list of reuse signatures is available in our replication database [2].

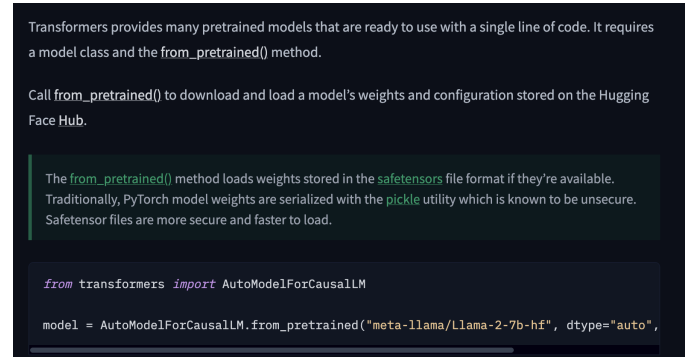


Fig. 1. Example of a model-loading function (`from_pretrained()`) in the `transformers` API documentation.

A.2 GitHub Query Strategy

We used the GitHub Search API (Search Code endpoint)³ to identify files containing reuse signatures. Each query combines import statements and usage patterns, with filters on programming language (Python), file size, and repository type (excluding forks). Compared with other tools used in previous work, such as Sourcegraph [1] and the GitHub Dependency Graph [3], the GitHub Search API offers broader coverage and more direct access to GitHub source code through Python SDKs. Although Sourcegraph offers the advantage of having no API rate limitation, it provides only a command-line interface and indexes a limited subset of public repositories, leading to fewer matches for equivalent search queries. The GitHub Dependency Graph, while useful for analyzing library-level dependencies, lacks a programmatic interface for code search. Therefore, it would require extensive web scraping and additional processes to connect with external tools for code search. In contrast, the GitHub Search API enables

1. <https://huggingface.co/docs>

2. <https://huggingface.co/docs/transformers/>

3. <https://docs.github.com/en/rest/search/search#search-code>

large-scale code-level searches for PTM reuse across repositories. Additionally, to improve coverage, we generated multiple query variants for each signature, including both import-style (e.g., `import transformers`) and from-style (e.g., `from transformers`) patterns.

To address the result limitation of the GitHub Search API, which returns a maximum of 1,000 results per query (10 pages with 100 results each), we implemented an adaptive query slicing strategy. This approach recursively divides the search space by file size until each subquery yields fewer than 1,000 results, ensuring complete coverage without violating API constraints. Each query combines both the import and usage signatures with additional filters for file size, programming language, and repository type.

An example query is shown below:

```
"import transformers" "from_pretrained(" in:file
↪ language:Python size:0..384000 NOT is:fork
```

Here, the query retrieves Python source files that include both an import of the transformers library and a corresponding PTM-loading call (`from_pretrained()`), restricted to files smaller than 384 KB—the maximum file size supported by the Search Code API. Using this approach, we obtained 730,891 reused files across 239,586 repositories that existed as of June-July 2025.

Appendix B

Static Validation and Repository-PTM Mapping

B.1 False Positive Filtering

To assess the reliability of the code search results, we manually inspected a statistically representative sample of 383 reused files, selected with a 95% confidence level and a 5% margin of error. The first author conducted the manual inspection, and representative cases of false positives and PTM–repository mappings were discussed with all authors to validate the categorization. This analysis revealed an overall false positive rate of 19% (73 files), indicating the need for additional validation beyond keyword-based matching.

We categorized false positives into four primary types:

- **DocString matches:** Reuse signatures appearing within comments or documentation strings (DocString), rather than executable code.
- **Unrelated method matches:** Function calls with identical names (e.g., `load()`, `from_pretrained()`) originating from unrelated libraries or contexts, leading to spurious matches.
- **Example or demonstration code:** PTM usage present in example scripts, tutorials, or demonstration files that do not reflect actual integration within the project.
- **Third-party code:** Files originating from external dependencies (e.g., `site-packages/`, vendored libraries) rather than project-authored source code.

These findings informed the design of our static analysis pipeline, which incorporates targeted filtering rules to eliminate such cases and improve the precision of PTM reuse detection. Our static analysis parse source code into an Abstract Syntax Tree (AST) and extract imports and function calls. We introduced filtering rules to handle all four

categories identified in the manual analysis. We evaluated these improvements using the same sampled set of 383 files described above. Compared to the baseline approach in [1], our method reduced false positives by 52.08%.

B.2 Repository-PTM Mapping

To build repository–PTM mappings, we collected open-source PTMs from the Hugging Face model hub⁴, the largest public registry of PTMs [4]. Using the official HfApi interface⁵, a Python wrapper of the Hugging Face Hub API⁶, we extracted model identifiers, names, and metadata to construct a searchable index. In total, we collected 1,805,282 PTMs from Hugging Face as of June 2025, representing an increase of about 1,350.88% over PTMTorrent [5] (January 2023) and 541.82% over PeaTMOSS [1] (August 2023).

To better understand how PTM identifiers are specified in practice, we manually inspected the same statistically representative sample of 383 reused files described in Section B.1. For this analysis, we considered only instances that were not identified as false positives (310). Based on this filtered set, we categorized model identifier specification into four levels of increasing complexity: *easy*, *normal*, *complicated*, and *impossible*.

- **Easy (32.26%):** Model identifiers are provided as direct string literals in function calls (e.g., `from_pretrained("bert-base-uncased")`).
- **Normal (20.65%):** Model identifiers are passed via variables, requiring basic data-flow tracking to resolve.
- **Complicated (23.23%):** Model identifiers are indirectly specified through configuration files (e.g., YAML/JSON), default function parameters, or user arguments with defaults.
- **Impossible (23.87%):** Model identifiers cannot be reliably resolved statically, such as when provided via user input without defaults, local file paths, or external runtime sources.

Examples.

```
model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L12-v2"
)
```

Listing 2. Easy case: direct string identifier.

```
model_name = "fixie-ai/ultravox-v0_3"
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

Listing 3. Normal case: identifier stored in a variable.

```
import json

config = json.load(open("config.json"))
model_path = config["model_name"]
tokenizer = AutoTokenizer.from_pretrained(model_path)
```

Listing 4. Complicated case: identifier from configuration or arguments.

Prior work [1] primarily handled the *easy* category. In contrast, our approach extends support to both *easy* and *normal* cases by introducing variable resolution, thereby covering a substantially larger portion of real-world PTM usage while remaining tractable for static analysis. Empirically, this exten-

4. <https://huggingface.co/>

5. https://github.com/huggingface/huggingface_hub

6. <https://huggingface.co/docs/hub/en/api>

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--model_path", type=str)
args = parser.parse_args()

model_path = args.model_path
tokenizer = AutoTokenizer.from_pretrained(model_path)
```

Listing 5. Impossible case: identifier from unresolved runtime input.

sion increases reused-file coverage by 64.00% compared to the previous work [1]. While the *complicated* and *impossible* cases represent a non-trivial portion of the sample, they require analysis beyond the capabilities of static analysis. Specifically, resolving identifiers in the *complicated* category often involves cross-file dependency tracking (e.g., parsing configuration files such as YAML/JSON), inter-procedural analysis to recover default parameter values, or partial execution to resolve dynamically constructed values. These tasks require building full program representations (e.g., control/data-flow graphs) or incorporating external artifacts, which introduces significant computational overhead and scalability challenges when applied to large-scale repositories. The *impossible* category further depends on runtime information that is inherently unavailable during static analysis, such as user-provided arguments without defaults, environment-specific paths, or external data sources. Accurately resolving these cases would require dynamic analysis or execution tracing, which is infeasible in our large-scale setting due to reproducibility, environment configuration, and resource constraints. Therefore, we focus on the *easy* and *normal* categories, which can be resolved reliably using static analysis while covering a substantial portion of real-world PTM usage.

To validate PTM reuse and extract precise mappings, we developed an AST-based static analysis pipeline extending the approach in [1]. The pipeline uses the Scalpel framework to parse Python source code into Abstract Syntax Trees (ASTs) and extract import statements and function calls. We enhanced the baseline approach with several improvements:

- **Single-pass analysis.** We redesigned the analyzer to process imports, variable assignments, and function calls in a single pass. This enabled tracking PTM identifiers stored in variables while preserving assignment order.

```
model_name = "bert-base-uncased"
model = AutoModel.from_pretrained(model_name)
```

Listing 6. Tracking PTM identifiers stored in variables.

- **Submodule-aware resolution:** Imports from nested modules (e.g., `transformers.models.auto`) were mapped to their base libraries.

```
from transformers.models.auto import AutoModel
model = AutoModel.from_pretrained("roberta-base")
```

Listing 7. Resolving submodule imports to base libraries.

Here, `AutoModel` originates from the `transformers` library, which must be correctly identified.

- **Conditional assignment handling.** The analysis supports conditional reassignment (e.g., `if-else`) by tracking multiple possible PTM identifiers per variable. Our method stored unconditional and branch-specific assignments sepa-

ately and retrieved all possible values before each call site, improving recall without constructing a full Control Flow Graph (CFG) [6].

```
if use_large:
    model_name = "bert-large-uncased"
else:
    model_name = "distilbert-base-uncased"
model = AutoModel.from_pretrained(model_name)
```

Listing 8. Handling conditional PTM selection.

- **Chained and nested call detection.** PTM loading was often embedded within chained method calls (e.g., `AutoModel.from_pretrained(...).to(...).half()`) can also be correctly recognized [7]. We extended function-call detection to handle such patterns. The pipeline correctly identified the PTM loading call despite subsequent transformations.

```
model = AutoModel.from_pretrained("gpt2").to(device).half()
```

Listing 9. Detecting chained PTM loading calls.

- **Attribute-based resolution.** PTM calls could be invoked through object attributes or class members rather than direct imports.

```
self.model = AutoModel
model = self.model.from_pretrained("bert-base-cased")
```

Listing 10. Resolving PTM calls through object attributes.

Our method resolved such indirect calls to recover the underlying PTM usage.

These enhancements significantly improved both precision and recall in identifying PTM reuse, while avoiding the computational overhead of full program analysis.

Appendix C

Final Filtering on Releases

C.1 Release-Based Filtering Criteria

To ensure that the final dataset reflects mature and systematic software engineering practices, particularly in release management [8], we applied a series of release-based filtering criteria adapted from prior studies on GitHub releases [9]–[12] and refined for PTM reuse.

Let $|\mathcal{R}|$ denote the number of releases in a repository, Δt the median release interval (in days), t_{last} the timestamp of the most recent release, and r_{semver} the proportion of releases following valid (strict or lenient) semantic versioning patterns. After excluding repositories without any releases ($|\mathcal{R}| \geq 1$), we applied the following filtering criteria:

- **Exclusion of draft and prerelease tags.** Draft and prerelease tags ($\neg(\text{draft} \vee \text{pre})$) were removed as they represent unstable or incomplete builds that do not reflect finalized software states.
- **Minimum release requirement.** Repositories with fewer than two releases ($|\mathcal{R}| < 2$) were excluded, as at least two releases are required to observe PTM changes across versions.
- **Release cadence filtering.** To remove irregular release practices [11], we excluded repositories whose

TABLE 1

Release-based filtering process showing the number of repositories and releases retained after each filtering step, along with the corresponding criteria.

Filtering Process	Criteria	# Repos remained	# Releases remained
Total	–	4,067	15,599
0) Excluding repos without release	$ \mathcal{R} \geq 1$	758 (-81%)	15,599
1) Excluding drafts/prereleases	$\neg(\text{draft} \vee \text{pre})$	727 (-0.7%)	14,087
2) Excluding repos with less than 2 releases	$ \mathcal{R} \geq 2$	558 (-4.2%)	13,918
3) Release frequency/cadence	$7 \leq \Delta t \leq 365$	383 (-4.3%)	6,525
4) Recency of releases	$t_{\text{last}} \geq 2025$	365 (-0.4%)	6,379
5) Naming patterns	$r_{\text{semver}} \geq 0.8$	341 (-0.6%)	6,242

median release interval fell outside the range $7 \leq \Delta t \leq 365$. We also filtered out repositories with extremely high release frequency (> 1 release/day) or very low activity (< 1 release/year).

- **Recency filtering.** Repositories without any releases in the most recent year ($t_{\text{last}} < 2025$) were excluded to ensure that only actively maintained projects were retained.

The cumulative impact of these filtering steps is summarized in Table 1, which reports the number of repositories and releases retained after each stage.

C.2 Semantic Versioning Classification

Here is the final step to ensure the quality of data based on releases. To ensure consistent identification of releases, we analyzed the naming patterns of all 6,379 remaining releases from the repositories that we had left. We manually performed a rigorous inspection of all tags to identify common naming styles, with heuristic classification rules and edge cases discussed and refined among all authors throughout the process. We defined three classification rules, which were then applied using regular expressions:

- **Strict Semantic Versioning:** Tags that fully comply with the MAJOR.MINOR.PATCH format defined by SemVer (e.g., 1.0.0, 2.1.3, 0.9.5, 3.2.1-beta.1, 2.1.0+build.5) [13]. These tags follow the standard structure, optionally including pre-release or build metadata.
- **Lenient Semantic Versioning:** Tags that approximate the SemVer structure but contain minor deviations in formatting or naming. Examples include prefixed versions (e.g., v1.0.0, release-2.3.1), partial versions (e.g., 1.0, v0.1), alternative separators (e.g., 1_7_5, 2-0-1), or embedded version identifiers (e.g., adapters1.0.1, bert-v2.4). These tags still encode version progression but do not strictly conform to the SemVer specification.
- **Violated Semantic Versioning:** Tags that do not represent version numbers or cannot be reliably interpreted as version identifiers. These include timestamps (e.g., 20230307081844, 2024-06-01), nightly or rolling builds (e.g., nightly-20250613, dev-latest), commit-based identifiers (e.g., build-abc123, snapshot-9f8e7d), or arbitrary labels (e.g., release, final, test, dummy). Such tags lack consistent version semantics and are excluded from analysis.

Applying these rules, 86.3% of releases were classified as strict, 11.9% as lenient, and 1.7% as violated. We excluded repositories whose combined proportion of strict and lenient tags fell below 0.8, as well as all releases violating SemVer. After applying these criteria, the dataset comprised 341 repositories, 832 reused files, and 6,242 releases, which were retained for analysis. To characterize the filtered dataset, we compared it with the broader pool of 4,067 confirmed PTM-reusing repositories using four activity measures: stargazers, issues, pull requests, and commits. Mann–Whitney U tests [14] indicate statistically significant differences across all measures ($p < 0.001$), with medium effect sizes measured using Cohen’s d (stargazers: -0.62 ; commits: -0.49 ; pull requests: -0.38 ; issues: -0.37) [15]. Detailed statistical results are reported in Table 2. These results suggest that the filtering criteria systematically favor more mature and actively maintained repositories, which aligns with our goal of ensuring reliable longitudinal analysis based on consistent release histories.

TABLE 2
Statistical comparison between full PTM-reusing repositories and release-filtered.

Metric	Med. (Before / After)	p -value	Cohen’s d
Size	9,460 / 21,791	5.43×10^{-12}	-0.00
Stargazers	26 / 203	1.13×10^{-43}	-0.62
Pull Requests	1 / 92	1.62×10^{-116}	-0.38
Commits	34 / 470	1.04×10^{-92}	-0.49
Open Issues	1 / 16	4.21×10^{-54}	-0.37

References

- [1] W. Jiang *et al.*, “PeaTMOSS: A Dataset and Initial Analysis of Pre-Trained Models in Open-Source Software,” in *Proceedings of the 21st IEEE/ACM International Conference on Mining Software Repositories (MSR 2024)*, vol. 1. New York, NY, USA: ACM, 2024, p. 431–443.
- [2] P. Banyongrakul, “Replication package for “when ai models become dependencies: Studying the evolution of pre-trained model reuse in downstream software systems”,” 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.19312246>
- [3] F. Pepe *et al.*, “How do Hugging Face Models Document Datasets, Bias, and Licenses? An Empirical Study,” in *Proceedings of the 32nd IEEE International Conference on Program Comprehension (ICPC 2024)*, no. iii. New York, NY, USA: ACM, 2024, pp. 370–381.
- [4] W. Jiang *et al.*, “An Empirical Study of Pre-Trained Model Reuse in the Hugging Face Deep Learning Model Registry,” in *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023)*. Piscataway, NJ, USA: IEEE, 2023, pp. 2463–2475.

- [5] —, “PTMTorrent: A Dataset for Mining Open-source Pre-trained Model Packages,” in *Proceedings of the 20th IEEE/ACM International Conference on Mining Software Repositories (MSR 2023)*. New York, NY, USA: ACM, 2023, pp. 57–61.
- [6] N. Dev, “The role of the control flow graph in static analysis,” October 2023, accessed: 2025-07-01. [Online]. Available: <https://nicolo.dev/en/blog/role-control-flow-graph-static-analysis/>
- [7] A. M. Keshk and R. Dyer, “Method Chaining Redux: An Empirical Study of Method Chaining in Java, Kotlin, and Python,” in *Proceedings of 20th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. Los Alamitos, CA, USA: IEEE, may 2023, pp. 546–557.
- [8] E. Laukkanen, M. Paasivaara, J. Itkonen *et al.*, “Comparison of release engineering practices in a large mature company and a startup,” *Empirical Software Engineering*, vol. 23, no. 6, pp. 3535–3577, 2018.
- [9] J. Coelho, M. T. Valente, L. L. Silva, and E. Shihab, “Identifying unmaintained projects in github,” in *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM 2018)*. New York, NY, USA: ACM, 2018.
- [10] S. D. Joshi and S. Chimalakonda, “RapidRelease - A dataset of projects and issues on github with rapid releases,” in *Proceedings of the IEEE/ACM 16th International Conference on Mining Software Repositories (MSR 2019)*, vol. 2019-May. Piscataway, NJ, USA: IEEE, 2019, pp. 587–591.
- [11] O. Kilic, N. Bowness, and O. Baysal, “Keep the Ball Rolling: Analyzing Release Cadence in GitHub Projects,” in *Proceedings of the 20th IEEE/ACM International Conference on Mining Software Repositories (MSR 2023)*. Piscataway, NJ, USA: IEEE, 2023, pp. 372–376.
- [12] D. Chakroborti, S. S. Nath, K. A. Schneider, and C. K. Roy, “Release conventions of open-source software: An exploratory study,” *Journal of Software: Evolution and Process*, vol. 35, no. 1, p. e2499, 2023.
- [13] T. Preston-Werner and Contributors, “Semantic versioning 2.0.0,” <https://semver.org>, 2023, accessed: Oct. 13, 2025.
- [14] H. B. Mann and D. R. Whitney, “On a test of whether one of two random variables is stochastically larger than the other,” *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.
- [15] M. J. Diener, “Cohen’s d,” in *The Corsini Encyclopedia of Psychology*. John Wiley & Sons, Ltd, 2010, p. 1.